

PRACTICAL: 05

Aim: To Implement and find the Time Analysis of a knapsack problem using dynamic programming.

Algorithm:

Algorithm

Input:

- * number of items n .
- * weight $w[i]$ and value $v[i]$ of each item.
- * knapsack capacity w .

Output

- * maximum value, that can be obtained.
- * selected items.

Step-1: start

Step-2: Read the number of item n and knapsack

Step-3: Read the weight $w[i]$ and value $v[i]$ of each item.

Step-4: Initialize the first row and first column of the DP table to 0.

Step-5: for each item $i=1$ to n , repeat step 6.

Step-6: for each capacity $j=1$ to w ,

if $w[i] < j$, calculate:

$$dp[i][j] = \max[v[i] + dp[i-1][j-w[i]], dp[i-1][j]]$$

otherwise: $dp[i][j] = dp[i-1][j]$

Step-7: The maximum value is stored in $dp[n][w]$.

Step-8: Trace the DP table from $dp[n][w]$ backward to find the selected items.

Step-9: Display the DP table, maximum value and selected item.

Step-10: stop.

Program / Code:

```
#include<iostream>
using namespace
std; class Knapsack
{ private:  int
w[10];  int v[10];
int n;  int W; int
dp[10][10]; public:
    void getInput() {      cout << "Enter number of
items: ";      cin >> n;      cout << "Enter
knapsack capacity: ";      cin >> W;      for(int i =
1; i <= n; i++) {          cout << "Item " << i << "
Weight and Value: ";          cin >> w[i] >> v[i];
        }
    }
    void solve() {
for(int i = 0; i <= n; i++) {
dp[i][0] = 0;
    }
    for(int j = 0; j <= W; j++){
dp[0][j] = 0;
        }
        for(int i = 1; i <= n;
i++){          for(int j = 1; j <=
W; j++){              if(w[i] <= j){
dp[i][j] = max(
v[i] + dp[i-1][j-w[i]],
dp[i-1][j]
                );          }
else{          dp[i][j] =
dp[i-1][j];
```

```
    }
    }
}

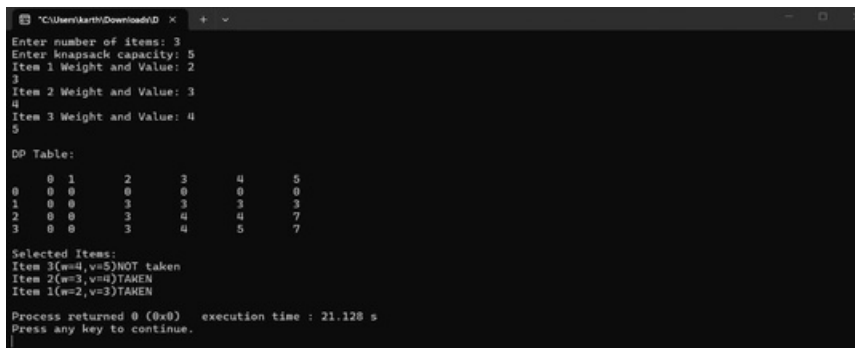
void printTable(){    cout
<< "\nDP Table:\n\n";
cout << "    ";    for(int j =
0; j <= W; j++){        cout
<< j << "\t";
    }
    cout << endl;    for(int i =
0; i <= n; i++){        cout << i
<< "    ";        for(int j = 0; j <=
W; j++){            cout <<
dp[i][j] << "\t";
        }
    cout << endl;
    }
}

void findItems(){    cout <<
"\nSelected Items: " << endl;    int j =
W;

    for(int i = n; i >= 1; i--){        if(dp[i][j] != dp[i-1][j]){            cout << "Item "
<< i << "(w=" << w[i] << ",v=" << v[i] << ")TAKEN" << endl;
            j = j - w[i];
        }
    }
else{
    cout << "Item " << i << "(w=" << w[i] << ",v=" << v[i] << ")NOT taken" << endl;
}
}
```

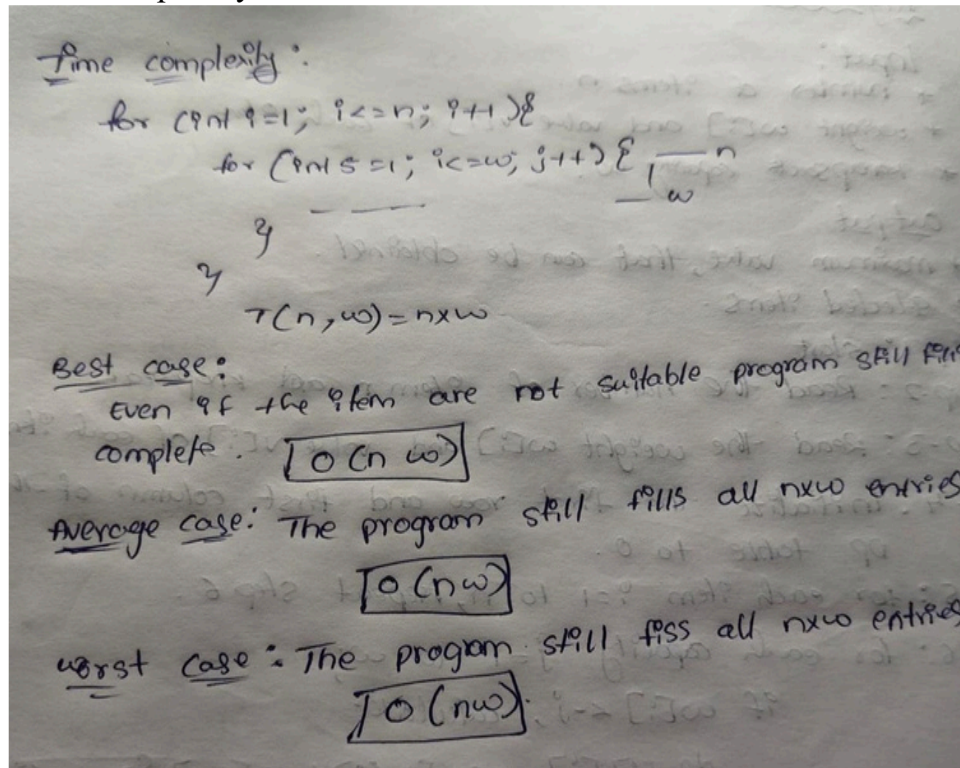
```
    }  
}  
  
void showResult(){    cout << "\nMaximum Value =  
" << dp[n][W] << endl;  
    } }; int  
main(){  
    Knapsack k;  
    k.getInput();  
    k.solve();  
    k.printTable();  
  
    k.findItems();  
    return 0; }
```

Output:



```
"C:\Users\karthi\Downloads\ID" x + v  
Enter number of items: 3  
Enter knapsack capacity: 5  
Item 1 Weight and Value: 2  
3  
Item 2 Weight and Value: 3  
4  
Item 3 Weight and Value: 4  
5  
  
DP Table:  
0 0 1 2 3 4 5  
0 0 0 0 0 0 0  
1 0 0 3 3 3 3  
2 0 0 3 4 4 7  
3 0 0 3 4 5 7  
  
Selected Items:  
Item 3(w=4,v=5)NOT taken  
Item 2(w=3,v=4)TAKEN  
Item 1(w=2,v=3)TAKEN  
  
Process returned 0 (0x0) execution time : 21.128 s  
Press any key to continue.
```

Time Complexity:



Conclusion:

The 0/1 Knapsack Problem was successfully solved using Dynamic Programming by building a table that considers, for every item, whether including or excluding it yields the better value. This bottom-up approach guarantees the optimal solution within the given capacity constraint. The algorithm runs in $O(nW)$ time and $O(nW)$ space, where n is the number of items and W is the knapsack capacity.